

SpecGen: Accelerating Agentic Kernel Optimization with Speculative Generation

Jihu Guo*
24110240025@m.fudan.edu.cn
Fudan University & Shanghai AI Lab
China

Sitian Lu*
sitianlu@sjtu.edu.cn
Shanghai Jiao Tong University &
Shanghai AI Lab
China

Tenghui Ma
24110240061@m.fudan.edu.cn
Fudan University & Shanghai AI Lab
China

Wei Gao†
csgaowei@ust.hk
Hong Kong University of Science and
Technology
China

Zhisheng Ye
yezshisheng@pku.edu.cn
Independent Researcher
China

Xingcheng Zhang
zhangxingcheng@pjlab.org.cn
Shanghai AI Lab
China

Dahua Lin
dhlin@ie.cuhk.edu.hk
The Chinese University of Hong Kong
& SenseTime Research
China

Abstract

Agentic kernel optimization automates manual GPU kernel tuning via iterative generation, validation, and profiling with reasoning LLMs, casting the optimization task as feedback-guided search. However, our workload characterization reveals three system-level inefficiencies that limit search efficiency: (1) long generation latency due to LLM reasoning, (2) insufficient profiling feedback, and (3) underutilized validation/profiling resources. Our key insight is that the ongoing reasoning generation exposes a window for producing additional candidate kernels before it completes, allowing the system to terminate reasoning early once a satisfactory kernel appears.

We present SpecGen, an agentic kernel optimization system with *speculative generation*. First, SpecGen forks non-reasoning generations at well-chosen trigger points in the reasoning trace to yield kernels, increasing the candidate kernel count per iteration. These kernels are validated and profiled in parallel with the ongoing reasoning, increasing profiling feedback, and keeping resources busy during generation. When a kernel meets the termination criterion, SpecGen terminates the reasoning generation early to reduce the generation latency. Second, SpecGen dynamically reallocates validation and profiling GPU pools based on the arrival rate and prioritizes requests to reduce profiling feedback latency under bursty speculative generation load. Furthermore, SpecGen utilizes spare memory of the validation/profiling GPUs as remote KV cache storage to eliminate prefix recomputation of speculative generations under limited memory budget. Experiments with two reasoning LLMs on H200

show that SpecGen reduces 1.68–1.82× end-to-end time over three baseline systems, while producing 1.58–1.98× profiling feedback, increasing resource utilization from 4.2–17.6% to 88.2–96.1%, and improving 1.24–1.91× kernel speedup under a fixed time and token budget.

1 Introduction

GPU kernels underpin every layer of modern LLM training and inference. Even a few percent improvement to attention, GEMM, or normalization kernels translates directly into reduced infrastructure cost [2, 7, 40]. However, kernel optimization is time and labor intensive, costing experts weeks of tuning kernel performance on specific GPUs [8, 9, 31]. Agentic kernel optimization automates the manual kernel tuning procedure with reasoning LLMs. It casts the kernel optimization process as a feedback-guided iterative search over a large kernel design space [5, 13, 27, 40]. Each iteration performs three phases in sequence: (1) *generation*: calling reasoning LLMs to generate candidate kernels; (2) *validation*: compiling and running these kernels against a reference kernel to check correctness and a first-cut speedup; and (3) *profiling*: measuring detailed performance metrics and adding them to the LLM’s context to inform subsequent generations. The objective of agentic kernel optimization is to *maximize kernel performance under a fixed time or token budget*.

However, the large kernel design space makes this search time- and resource-costly, with a kernel optimization task often taking tens of hours or even days [26, 27, 40]. To understand where the budget goes, we conduct a comprehensive workload characterization (see §3) of agentic kernel optimization across 10 KernelBench [28] tasks on an H200 testbed

*These authors contributed equally to this research.

†Corresponding author.

with two state-of-the-art open-source reasoning LLMs, GLM-5.1 [14] and DeepSeek-V4-Pro [10]. Our workload characterization identifies three system-level inefficiencies that limit search efficiency: **(C1)** long generation latency: the generation phase accounts for 70–99% of iteration time, directly capping the iteration count under fixed time budget; **(C2)** insufficient profiling feedback: about 59–64% of iterations consume GPU hours without producing profiling feedback to drive the next iteration’s optimization; and **(C3)** underutilized validation and profiling resources: validation and profiling GPUs achieve only 4.2–17.6% utilization. They remain idle during the generation phase because candidate kernels emerge only after generation completes, and many candidates fail validation before profiling.

To address these inefficiencies, prior systems adopt different harness engineering strategies, such as best-of- K [5, 40], multi-agent [26], and evolutionary algorithms [13, 27]. Despite their strategies, these systems share a common pattern. They dispatch multiple reasoning generations per iteration to increase the candidate count and produce more profiling feedback, but this roughly multiplies token cost by K . A strawman alternative is to dispatch non-reasoning generations to reduce token cost. However, non-reasoning generations rarely produce valid or performant kernels (see Table 2). Moreover, prior systems do not reduce generation latency since candidate kernels become actionable only after the generation finishes, and validation/profiling resources remain idle during the reasoning process of the generation phase.

Our key insight is that the reasoning process exposes a window for producing additional candidate kernels before the reasoning generation completes (§4.2). Since non-reasoning generations are faster than reasoning generations, the system can dispatch multiple non-reasoning generations in parallel with the ongoing reasoning generation to increase the candidate kernel count. To improve the kernel performance of non-reasoning generations, we condition non-reasoning generations on the prefix of the reasoning output. With this conditioning, the non-reasoning generations can produce valid and performant kernels before the reasoning generation completes (see Table 2). Our further experiments show that prefix conditioning can even generate kernels with higher speedups than the average speedup of previously generated kernels (see Figure 6). This provides three benefits: **(1)** shorter generation latency by terminating the reasoning process early when a satisfactory candidate emerges **(C1)**; **(2)** more profiling feedback through generating more candidate kernels per iteration **(C2)**; **(3)** higher validation/profiling utilization by keeping validation/profiling resources busy during generation **(C3)**. We empirically show that a pragmatic termination criterion reduces generation latency without compromising kernel performance (see §8.9). Building on this insight, we introduce *speculative generation*, which forks non-reasoning generations conditioned on the

reasoning prefix. Because each fork shares the cached reasoning prefix, speculative generation incurs low token overhead (see §8.7). Moreover, speculative generation operates at the iteration level and is orthogonal to token-level speculative decoding [4, 19]. The two techniques can be combined to further improve system efficiency.

However, realizing speculative generation introduces two system-level challenges. **First**, *how* to fork and *when* to terminate: the system must identify useful conditioning context from a noisy reasoning trace and decide how many speculative generations to launch. Too few forks leave validation/profiling resources idle, while too many overload the queues and delay profiling feedback. Terminating too early sacrifices kernel performance, while terminating too late saves negligible generation time. **Second**, *how* to manage bursty speculative-generation load. The legacy static “one GPU per kernel” partitioning cannot handle this burstiness. It leaves some GPUs idle while others queue, so resource allocation must adapt as bursts arrive. The system also needs request prioritization to bound profiling feedback latency and memory management to support bursty speculative generations in local LLM deployments.

We present SpecGen, an agentic kernel-optimization system that wraps a user-specified LLM, prompt, and search algorithm with two cooperating components. **(1) SpecController** monitors the reasoning model’s output stream and parses it to detect trigger signals, such as kernel-design decisions, fenced code blocks, and closed kernel bodies. On a clean trigger, SpecController concatenates the iteration’s prompt with the reasoning prefix and dispatches K non-reasoning speculative generation requests. The number of forks adapts to the available validation/profiling resource capacity, exposing enough candidates to keep GPUs busy without overloading the queues. Once a speculatively generated kernel exceeds the average speedup of previously generated kernels, SpecGen terminates the reasoning generation early to reduce the generation latency. This threshold avoids terminating on weak speculative kernels while still allowing frequent early terminations. **(2) ElasticScheduler** manages one elastic GPU pool, dynamically split between validation and profiling. Each request takes any free GPU in its pool, so no GPU idles while another queues up. Between iterations, ElasticScheduler reallocates the two pools based on the arrival rates of the previous iteration. Within an iteration, the validation queue is served last-arrival-first. Later candidates carry more reasoning prefix and are more likely to validate. The profiling queue is served FIFO, so any validated kernel returns feedback fast and SpecController decides on the freshest signal. We also observe that validation/profiling GPUs retain substantial unused memory even under bursty speculative load. We therefore repurpose this spare memory as remote storage for the reasoning prefix cache, eliminating prefix recomputation of speculative generations.

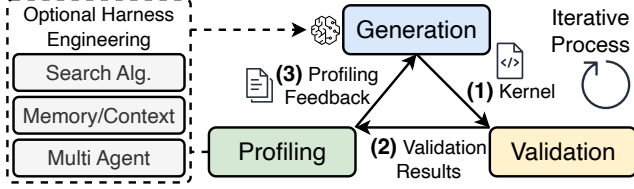


Figure 1. The three phases of agentic kernel optimization iteration: generation, validation, and profiling.

Across KernelBench Level 1/2/3 tasks with two reasoning LLMs on H200, SpecGen consistently improves the efficiency of agentic kernel optimization against CudaForge [40], AlphaEvolve [27], and KernelAgent [26]. Compared with these baselines, SpecGen reduces end-to-end execution time by 1.68–1.82 $\times$ and increases profiling feedback by 1.58–1.98 $\times$. It also improves final kernel speedup by 1.24–1.91 $\times$, showing that shorter E2E time does not come at the cost of kernel performance. Finally, SpecGen increases validation/profiling utilization from 4.2–17.6% to 88.2–96.1%.

This paper makes the following contributions:

- **Characterization.** We conduct a comprehensive workload characterization of agentic kernel optimization and identify three system-level inefficiencies.
- **Insight.** We empirically establish that the reasoning process exposes a window for speculative generations conditioned on the reasoning prefix.
- **System.** We implement SpecGen, comprising a Spec-Controller and an ElasticScheduler that realize the above insight to address the three inefficiencies without sacrificing kernel performance.
- **Evaluation.** We evaluate SpecGen on 20 KernelBench Level 1/2/3 tasks across two reasoning LLMs on H200 against three state-of-the-art agentic kernel optimization systems, with detailed results in §8.

2 Background

GPU kernel optimization is an iterative and verifiable task. Realizing a competitive kernel typically requires experts weeks per operator on a given GPU [7, 13, 22, 27].

Agentic Kernel Optimization iteratively performs three phases as shown in Figure 1 to generate and optimize kernels. (1) Generation invokes the LLM on the accumulated context to produce a candidate kernel (e.g., CUDA/C++ [28] or Triton [21]). (2) Validation checks the candidate kernel for compilation, execution, and output correctness against a reference [21, 28]. Any failure (compile error, runtime error, or numerical mismatch) is merged into the LLM’s context to guide the next iteration’s correction. Only kernels passing all three checks proceed to profiling. (3) Profiling measures the candidate kernel’s detailed performance metrics using NVIDIA Nsight Compute (NCU). The resulting metrics are merged into the LLM’s context as feedback to guide subsequent generations. Both validation and profiling require exclusive GPU access for measurement accuracy, since GPU

Table 1. Short names (T1–T10) of the ten KernelBench kernels widely adopted in LLM training and inference.

Abbr.	Task	Abbr.	Task
T1	HingeLoss	T6	Upper-tri. Matmul
T2	3D tensor Matmul	T7	Lower-tri. Matmul
T3	4D tensor Matmul	T8	$A^T B$ Matmul
T4	Diagonal Matmul	T9	AB^T Matmul
T5	Symmetric Matmul	T10	$A^T B^T$ Matmul

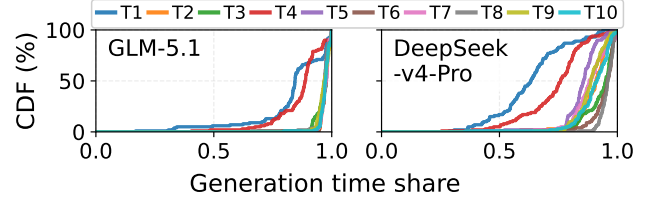


Figure 2. Per-iteration generation time share across ten KernelBench kernel optimization tasks per model.

sharing would distort both the speedup measured against the reference kernel and the profiled performance metrics. Due to this exclusivity, existing frameworks statically dedicate one GPU per kernel to run its validation and profiling [20, 40], a partitioning we refer to as “one GPU per kernel”.

Harness Engineering Approaches are also explored for agentic kernel optimization, including multi-agent collaboration [33, 37, 40] and memory/context management [5, 11, 30], evolutionary search [13, 26, 27, 32, 34], and verifier-guided search [16, 17, 36] to improve the kernel performance (Figure 1 left). These works differ in their agentic optimization algorithms but still rely on the same three-phase iteration: invoking the LLM to generate kernels, validating their correctness against a reference, and profiling them for performance metrics. Despite the rapidly evolving algorithmic frontier, the system-level efficiency of existing harness engineering approaches has received little attention. To analyze it, we characterize the agentic kernel optimization workload in §3.

3 Workload Characterization

We characterize the workload of agentic kernel optimization on top of the iterative refinement framework from KernelBench [28], an efficient agentic kernel optimization framework for improving kernel performance. We run two reasoning LLMs, GLM-5.1 [14] and DeepSeek-V4-Pro [10], on 10 KernelBench [28] kernel optimization tasks widely used in LLM training and inference (Table 1, 100 iterations per LLM per kernel), with H200 GPUs dedicated per kernel for validation and profiling. The resulting traces reveal three system-level inefficiencies, which we analyze in §3.1–§3.3.

3.1 Long Generation Latency

We define the per-iteration *generation time share* as the fraction of iteration time spent in the generation phase. Figure 2 reports its empirical CDF across both reasoning models and the ten KernelBench tasks. Under GLM-5.1, the task-level

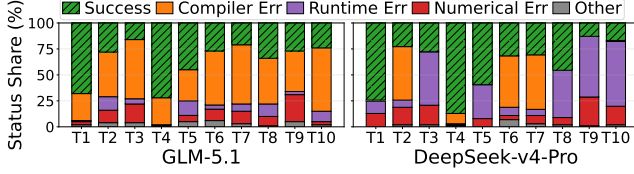


Figure 3. Distribution of iteration status across 100 iterations per kernel with two models. An iteration is *Success* iff its emitted kernel compiles, runs, and matches the reference.

75th-percentile (P75) of this share falls between 92–99%, with most matmul variants clustered near 98%. For DeepSeek-V4-Pro, P75 spans 70%–98%, with HingeLoss (T1) as a low outlier at 70% and the remaining matmul workloads at P75 between 81% and 98%. Across both models and the vast majority of iterations, LLM generation thus accounts for the bulk of each iteration’s wall-clock, leaving validation and profiling as second-order time costs on the critical path. In absolute terms, generation averages 706.9 s and 522.6 s per iteration on the two models, an order of magnitude above validation (22.9 s and 59.0 s) and profiling (26.5 s and 26.6 s).

Reason. Kernel optimization is an intrinsically multi-axis problem. Each iteration must reconcile hardware constraints, I/O and precision requirements, and evolving compute-versus-memory bottlenecks against the previous iterations’ profiling feedback. Models therefore need to reason longer to produce a kernel, leading to longer generation latency.

Implication 1. Long generation latency limits the iteration count under any fixed time budget, motivating a scheme to reduce the generation latency.

3.2 Insufficient Profiling Feedback

Profiling feedback refers to the NCU profiling results of a valid kernel, which the agent uses to guide the next iteration’s optimization. However, Figure 3 shows that 36.3% of iterations under GLM-5.1 and 40.7% under DeepSeek-V4-Pro produce kernels that pass both validation and profiling. On seven of the ten tasks, fewer than half the iterations yield profiling feedback. Failure modes vary by model. GLM fails predominantly at compile time on the harder matmul variants, indicating brittle code shape and syntax (e.g., templates, kernel signatures, includes). DeepSeek shows more runtime failures, suggesting it compiles more aggressive variants whose indexing, memory access, or launch configurations are unsafe. Numerical mismatches recur on transpose and triangular matmul for both models, pointing to subtle layout or accumulator bugs that pass static checks but produce incorrect outputs.

Reason. The insufficient profiling feedback stems from a gating effect between validation and profiling. Profiling measures real performance metrics on the GPU using NCU, and is meaningless for incorrect kernels. The agent therefore only profiles kernels that pass validation. The remaining iterations fail validation, never reaching the profiler.

Table 2. Best speedups over the KernelBench reference from 100 non-reasoning generations w/o and w/ conditioning on *reasoning prefixes* with two models. $\times$: no available kernels.

Method	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
GLM w/o	$\times$	0.44	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	$\times$	1.00
GLM w/	8.54	2.79	0.79	58.16	4.21	3.41	2.90	4.69	4.00	4.25
DSv4 w/o	$\times$	$\times$	$\times$	57.27	0.36	0.81	0.35	0.44	$\times$	0.02
DSv4 w/	8.76	0.68	0.88	61.32	4.04	6.35	2.20	3.51	1.33	0.73

Implication 2. The insufficient profiling feedback forces much of the budget into failed candidates rather than performance-improving exploration. This motivates mechanisms that increase the number of valid candidates reaching profiling.

3.3 Underutilized Validation/Profiling Resources

Under the “one GPU per kernel” partitioning introduced in §2, the validation/profiling resource pool is severely underutilized. Figure 4 shows the in-flight request counts for the three phases over the first 10,000 seconds of agentic kernel optimization. We observe that the validation/profiling resource pool sustains only 0.14–1.68 concurrent requests per second on average, one to two orders of magnitude below the 8.19–9.62 concurrent generation requests over the same time window. The resource utilization is 4.7% on GLM and 11.3% on DeepSeek. The validation and profiling load is thus one to two orders of magnitude lower than generation.

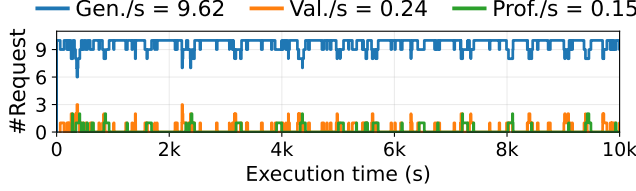
Reason. First, candidate kernels emerge only after the generation completes, so validation GPUs remain idle during most of the generation phase. Second, many candidates fail validation and never reach profiling, leaving profiling GPUs with even fewer requests.

Implication 3. Validation and profiling GPUs sit chronically idle during the generation phase, leaving GPU resources underutilized. This motivates mechanisms that make candidate kernels available while reasoning generation is still running.

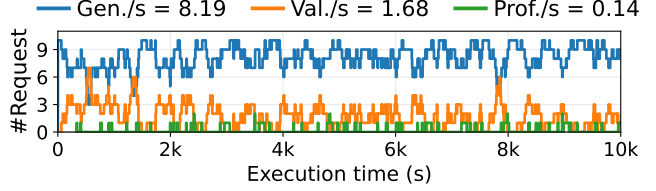
4 Design Insight and Challenges

4.1 Limitations of Prior Systems

As illustrated in §3, the baseline iterative refinement framework [28], which performs one generation per iteration (see Figure 5(a)), suffers from three system-level inefficiencies. An intuitive approach is to dispatch multiple generations per iteration to improve search efficiency. However, (1) dispatching multiple *reasoning generations* [5, 13, 26, 27, 40] roughly multiplies the token cost by K (see Figure 5(b)). The candidate kernels become actionable only after the reasoning process finishes, resulting in significant idle time for validation/profiling resources. (2) Dispatching multiple *non-reasoning generations* reduces both generation latency and token cost but rarely produces valid or performant kernels. Table 2 shows that 8/10 tasks on GLM and 4/10 tasks on DeepSeek fail to produce any valid kernels after 100 iterations of non-reasoning generations.



(a) GLM-5.1.



(b) DeepSeek-V4-Pro.

Figure 4. In-flight request counts for the three pipeline phases (generation, validation, profiling) over the first 10,000 seconds of execution, with 10 agent workflows and 10 exclusive-mode validation/profiling GPUs.

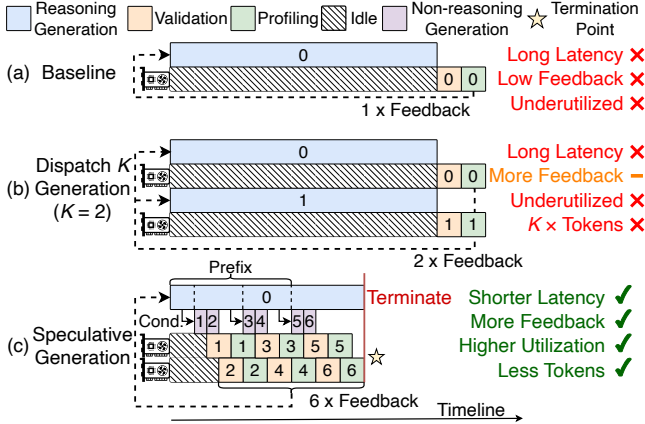


Figure 5. Comparison of existing techniques in agentic kernel optimization and speculative generation.

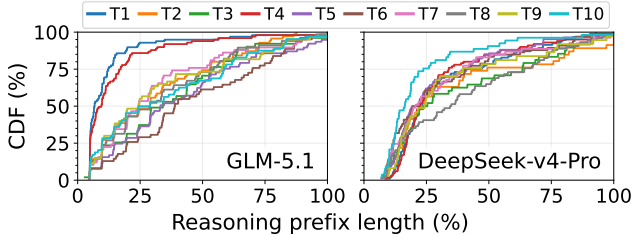


Figure 6. CDF of the shortest reasoning prefix length for generating a kernel with higher speedups than the average speedup of previously generated kernels across both models.

4.2 Insight: Speculative Generation

Our key insight is that the reasoning process exposes a window for producing additional candidate kernels with *reasoning prefix conditioning*. Since non-reasoning generations are faster than reasoning generations, the system can dispatch multiple non-reasoning generations in parallel with the ongoing reasoning generation to increase the candidate kernel count. To improve the kernel performance of non-reasoning generations, we condition non-reasoning generations on the prefix of the reasoning output. Table 2 shows that with this conditioning, non-reasoning generations produce valid kernels on all tasks and achieve higher speedups. Moreover, because this conditioning shares the reasoning prefix with the reasoning generation, speculative generations can reuse the prefix cache and reduce token cost.

We further explore early termination opportunities with prefix conditioning in Figure 6. We record the shortest reasoning prefix that produces a kernel faster than the average of previously generated kernels. Across GLM-5.1 and DeepSeek-V4-Pro, the prefix length ranges from 14% to 70% of the full reasoning trace. These results show that prefix conditioning can provide useful context for non-reasoning generations to produce performant kernels before the full reasoning trace completes. Once a satisfactory kernel appears, the system can terminate the reasoning generation early to reduce generation latency. We empirically show that a pragmatic termination criterion can reduce generation latency without sacrificing kernel performance (see §8.9).

This insight motivates an iteration-level speculative generation scheme. Compared with the prior approaches in Figure 5(b), speculative generation in Figure 5(c) introduces three benefits: (1) shorter generation latency by enabling early termination when a forked kernel meets the termination criterion (C1); (2) more profiling feedback via generating more candidate kernels per iteration (C2); (3) higher validation/profiling utilization by keeping resources busy during the generation phase (C3).

4.3 Challenges of Speculative Generation

Realizing speculative generation poses two challenges.

Challenge 1. How to fork and when to terminate. Reasoning traces contain noise, such as self-talk and repetition, that disturb the parsing of useful context for non-reasoning generations. The system must identify useful conditioning context from a noisy reasoning trace and decide how many speculative generations to launch. Too few forks leave validation/profiling resources idle, while too many overload the queues and delay profiling feedback. The system should also carefully choose the termination criterion. Terminating too early sacrifices kernel performance, while terminating too late saves negligible generation time.

Challenge 2. How to manage bursty speculative generation load. Speculative generation reshapes the validation and profiling load from sporadic to bursty. The legacy static “one GPU per kernel” partitioning cannot handle this burstiness. It leaves some GPUs idle while others queue, so resource allocation must adapt as bursts arrive. Bounding

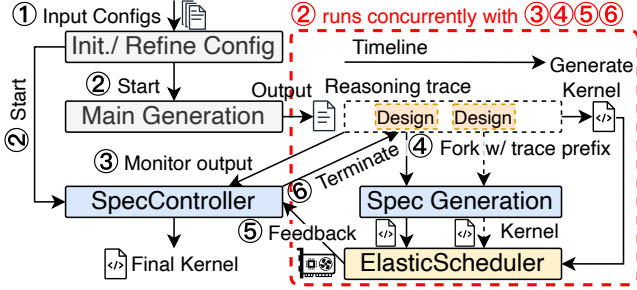


Figure 7. System overview of SpecGen.

the profiling feedback latency also requires request prioritization. For local LLM deployments, bursty speculative generations further introduce additional memory overhead that the system needs to manage.

5 System Overview

Overview. We introduce SpecGen, an agentic kernel optimization system comprising *SpecController* and *ElasticScheduler*. *SpecController* utilizes the ongoing reasoning generation as a window for speculative generation. It forks non-reasoning generations conditioned on the reasoning prefix while keeping the reasoning generation as a fallback. Once a satisfactory kernel appears, *SpecController* terminates the reasoning generation. *ElasticScheduler* dynamically reallocates the validation/profiling resource pool, prioritizes validation and profiling requests, and uses spare memory on validation/profiling GPUs as remote KV-cache storage for reasoning prefixes. SpecGen requires *no changes* to the underlying LLM or search algorithm. It reduces generation latency, produces more profiling feedback, and keeps validation/profiling resources busy.

Workflow. One iteration of SpecGen proceeds in the six steps below. **(1) Input configuration.** The user supplies three inputs: the reasoning LLM that will drive optimization, the prompt template that describes the target kernel and feedback format, and the search algorithm (e.g., evolutionary search [26, 27, 32], best-of- N [5, 20, 40]) that governs how optimization status is updated between iterations. Optionally, the user supplies a termination criterion for early stopping. **(2) Start main generation and SpecController.** SpecGen starts the main reasoning generation. At the same time, *SpecController* attaches to the main generation’s reasoning output stream as a monitor. The main generation remains live while steps (3)–(6) execute, so these steps run concurrently with step (2). **(3) Monitor output.** *SpecController* monitors the main generation’s reasoning trace and detects when the trace has committed to concrete kernel design, such as a tile-shape choice, a parallelization plan, or a fenced CUDA code block. These kernel-design decisions serve as trigger signals for non-reasoning speculative generations. **(4) Fork speculative generation.** When *SpecController* emits a trigger signal, it concatenates the iteration’s prompt with the reasoning trace prefix to form a speculative prompt.

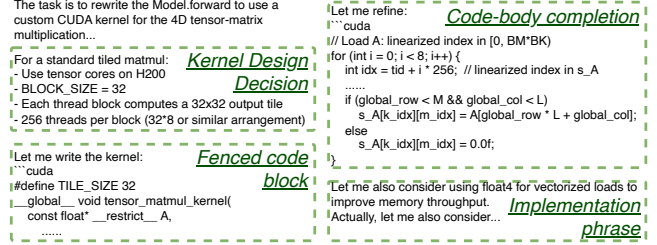


Figure 8. A reasoning trace of GLM-5.1 on the 4D tensor Matmul task. Trigger signals are highlighted in red.

It forks speculative generations with this speculative prompt. As each fork emits a candidate kernel, *SpecController* dispatches the kernel to *ElasticScheduler* for validation and profiling. *SpecController* maintains a history of kernels and their speedups to evaluate the termination criterion. **(5) Elastic scheduling for validation and profiling.** *ElasticScheduler* receives validation/profiling requests from *SpecController* and returns validation/profiling feedback to *SpecController*. It dynamically partitions the GPU pool between validation and profiling to absorb bursty arrivals, keep resources busy, and bound profiling feedback latency. **(6) Termination and output final kernel.** *SpecController* analyzes the validation/profiling feedback to decide when to terminate the main reasoning. If a speculative kernel meets the termination criterion, *SpecController* terminates the main generation and outputs this kernel as the final result. If no early termination fires, the main reasoning generation remains a fallback, and SpecGen selects the best kernel found in the iteration.

6 Methodology

This section details the two components of SpecGen: *SpecController* (§6.1), which decides when and how to fork speculative generations, and *ElasticScheduler* (§6.2), which dynamically resizes the resource pool and orchestrates concurrent validation and profiling requests.

6.1 SpecController

Algorithm 1 shows when *SpecController* forks speculative generations and when it terminates the reasoning generation. *SpecController* monitors the reasoning generation’s output. At selected trigger points, it concatenates this iteration’s prompt with the reasoning prefix to form a speculative prompt. Conditioned on this speculative prompt, *SpecController* forks non-reasoning generations and dispatches their emitted kernels to validation and profiling while the reasoning generation continues as a fallback. Once a kernel meets the termination criterion, *SpecController* terminates the reasoning generation and all pending speculative generations to reduce generation latency. We next detail when *SpecController* triggers speculative generations (§6.1.1) and when it terminates the reasoning generation (§6.1.2).

6.1.1 Trigger Speculative Generation. *SpecController* uses kernel-design decisions in the reasoning trace as trigger

Algorithm 1 SpecController core loop.

Require: Reasoning LLM M , prompt P_0 , search algorithm $\mathcal{A}$, resource capacity C , #iterations N

```

1:  $H \leftarrow \{0\}$ ;  $P \leftarrow P_0$      $\triangleright$  Speedup history; initial prompt
2:  $k^* \leftarrow \perp$                  $\triangleright$  Best kernel found so far
3: for  $i = 1$  to  $N$  do
4:    $et \leftarrow \text{false}$            $\triangleright$  Early termination flag
5:    $\mathcal{G} \leftarrow \text{stream of } M(P)$   $\triangleright$  Reasoning generation
6:   while  $\mathcal{G}$  is not terminated do
7:     if  $\mathcal{G}$  emits a trigger signal or GPU is idle then
8:        $\pi \leftarrow \text{prefix of } \mathcal{G}$ 
9:        $P_s \leftarrow \text{CONCAT}(P, \pi)$   $\triangleright$  Speculative prompt
10:       $K \leftarrow \max(1, \min(C.val, C.prof))$ 
11:      Fork  $K$  spec. generations with prompt  $P_s$ .
12:    end if
13:    for each spec. generation  $G_s$  with kernel  $k_s$  do
14:       $speedup_s \leftarrow \text{measured speedup of } k_s$ 
15:       $\tau \leftarrow \text{mean}(H)$ 
16:       $H \leftarrow H \cup \{speedup_s\}$ 
17:      if  $speedup_s > \tau$  then
18:         $k^* \leftarrow k_s$ ;  $et \leftarrow \text{true}$ 
19:        Terminate  $\mathcal{G}$  and all pending forks;
20:        break
21:      end if
22:    end for
23:  end while
24:  if  $et$  is false then
25:    Update  $k^*$  with the best found kernel.
26:  end if
27:   $P \leftarrow \mathcal{A}(k^*, H)$   $\triangleright$  Update prompt for next iteration
28: end for
29: return  $k^*$                  $\triangleright$  The best found kernel

```

signals for speculative generations. Figure 8 shows an example of the reasoning trace from GLM-5.1 on the 4D tensor Matmul task. During the reasoning process, we observe that the LLM commits concrete kernel-design decisions. As reasoning progresses, the LLM implements these decisions in fenced code blocks or completed kernel bodies. Finally, the LLM generates the final code based on the reasoning results. Based on this observation, SpecController uses an empirical parser to detect these trigger signals. The parser recognizes four classes of trigger signals. (1) *Kernel-design decisions*. Concrete kernel-design decisions, such as tile shapes, tile sizes, and specific instructions. (2) *Fenced code blocks*. Complete code blocks fenced with language tags such as `cuda`, `cpp`, or `python`. (3) *Kernel-body completion*. A partial kernel body, such as a `__global__` function with a complete signature and brace-balanced body. (4) *Implementation phrases*. Natural-language phrases that announce implementation, such as “Let me implement...”, “Here is the plan...”. We derive these trigger structures from 38,745 reasoning traces

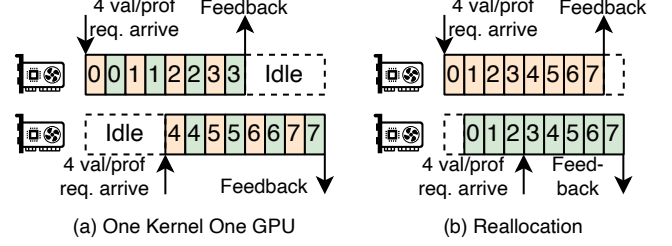


Figure 9. Reducing GPU idle with reallocation.

generated by GLM-5.1 and DeepSeek-V4-Pro, and implement the parser with regular expressions.

Once SpecController detects these trigger signals, it concatenates the iteration’s original prompt and the reasoning trace prefix to construct a speculative prompt. Then SpecController forks non-reasoning generations with this speculative prompt. To avoid speculative-generation starvation when no trigger is detected, SpecController also forks from the current reasoning prefix whenever validation/profiling resources become idle. Each fork event launches K speculative generations, where K is determined by the resource capacity C (Algorithm 1, Line 10). This lets SpecController launch enough speculative generations to keep resources busy without queuing too many requests and wasting tokens.

6.1.2 Early Termination. SpecController terminates the reasoning generation once a speculative kernel meets the predefined termination criterion. After a speculative generation emits a candidate kernel, SpecController sends it to ElasticScheduler (§6.2) for validation and profiling. For each valid kernel, ElasticScheduler returns the measured speedup over the KernelBench reference. SpecController maintains a history H of previously profiled kernel speedups and compares each new kernel’s speedup against this history.

In our experiments, we use the mean speedup of previously profiled kernels, $\text{mean}(H)$, as the termination threshold. When a speculative kernel exceeds this threshold, SpecController terminates the reasoning generation and cancels all pending speculative generations. Otherwise, the reasoning generation continues as a fallback. This adaptive threshold maintains termination frequency without sacrificing kernel performance. Figure 13 empirically confirms the efficiency of this termination criterion. As optimization progresses, $\text{mean}(H)$ rises with the performance of previously found kernels. At the same time, it is less sensitive than $\text{max}(H)$ to a single outlier, which could otherwise suppress early termination for many later iterations. For example, on the Diagonal Matmul task, a kernel reaches $56\times$ speedup over the reference in iteration 5 and remains the best kernel until iteration 72 (Figure 13). If the threshold were $\text{max}(H)$, early termination would rarely fire during this interval, losing generation-latency savings without improving the best

Algorithm 2 ElasticScheduler core loop.

Require: Total GPU pool G

```
1:  $Q_v \leftarrow \emptyset; Q_p \leftarrow \emptyset$   $\triangleright$  Request queues
2:  $L_v \leftarrow 0; L_p \leftarrow 0$   $\triangleright$  Max queue lengths
3: for each iteration  $i$  do
4:    $(G_{val}, G_{prof}) \leftarrow \text{ALLOCATE}(L_v, L_p, G)$   $\triangleright$  Reallocation
5:    $L_v \leftarrow 0; L_p \leftarrow 0$   $\triangleright$  Reset max queue lengths
6:   async
7:     Queue requests to  $Q_v$  (LAF) and  $Q_p$  (FIFO).
8:      $L_v \leftarrow \max(L_v, |Q_v|); L_p \leftarrow \max(L_p, |Q_p|)$ 
9:     Serve requests with  $G_{val}$  and  $G_{prof}$  on idle GPUs.
10:  end async
11:  Terminate in-flight requests and clear  $Q_v$  and  $Q_p$ .
12: end for
```

kernel. Instead, using $\text{mean}(H)$ as the threshold allows SpecController to reduce generation latency through more frequent early termination (Figure 10). SpecGen also exposes interfaces for user-supplied termination criteria.

6.2 ElasticScheduler

Algorithm 2 illustrates the core loop of ElasticScheduler. At the beginning of each iteration, ElasticScheduler calls `ALLOCATE` to reallocate the validation/profiling GPU split (G_v, G_p) based on the maximum validation and profiling queue lengths, L_v and L_p , from the previous iteration. ElasticScheduler receives validation and profiling requests from SpecController and serves them on the corresponding GPU split with two priority queues. At the iteration boundary, SpecController consumes only the validation/profiling results that have already returned. ElasticScheduler then aborts remaining requests from the finished iteration and clears both queues, so speculative tails never delay the next iteration. Finally, ElasticScheduler repurposes unused GPU memory in the validation/profiling GPUs as remote KV-cache storage for the reasoning prefix. We next detail the resource reallocation (§6.2.1), the priority queues (§6.2.2), and the remote cache of reasoning prefixes (§6.2.3).

6.2.1 Resource Reallocation. ElasticScheduler divides the total GPU pool into two splits: validation GPUs and profiling GPUs. The number of GPUs in each split is determined by the max queue length L_v and L_p from the previous iteration. Specifically, when $L_v + L_p = 0$, ElasticScheduler uses an even split. Otherwise, it sets $G_{prof} = \min(G - 1, \max(1, \lceil G \cdot \frac{L_p}{L_v + L_p} \rceil))$, $G_{val} = G - G_{prof}$. With this reallocation, ElasticScheduler can absorb bursty speculative-generation load and reduce GPU idle. Figure 9 shows an example of the resource reallocation. In this example, the static one-GPU-per-phase allocation keeps one GPU assigned to validation and one GPU assigned to profiling. Because validation and profiling requests arrive at different times, one split can queue requests while the other split remains idle.

ElasticScheduler reallocates the two GPUs according to the previous iteration’s queue pressure, allowing idle GPUs to move toward the heavier-loaded phase.

6.2.2 Reducing Profiling Feedback Latency. ElasticScheduler prioritizes latest-arrival validation requests (LAF) and earliest-arrival profiling requests (FIFO). Later validation requests are produced from longer reasoning prefixes. They are therefore conditioned on more design information than earlier requests. Later candidates are therefore more likely to contain complete design information and to meet the early-termination threshold. LAF intentionally favors fresher candidates within an iteration. Older validation requests may be skipped at the iteration boundary, but this does not block progress because the reasoning generation remains a fallback and the next iteration starts with a cleared queue. For profiling requests, FIFO avoids delaying an already validated kernel behind later arrivals. Because every profiled kernel provides usable feedback, serving the oldest validated kernel first reduces feedback latency without favoring speculative freshness.

6.2.3 Remote Cache of Reasoning Prefixes. Speculative generations share the reasoning prefix with the main generation. However, keeping all prefix KV caches in local serving memory can exceed the memory budget when multiple speculative generations are active. ElasticScheduler therefore uses spare memory on validation/profiling GPUs as remote KV-cache storage. Validation and profiling consume little GPU memory in our workload, even under bursty speculative load, so the remote cache uses otherwise idle memory without reducing the memory available to validation or profiling execution.

When local serving memory approaches its capacity limit, ElasticScheduler migrates KV caches of suspended reasoning prefixes to the remote GPU pool instead of discarding them. When a speculative generation later resumes from the same prefix, it restores the cached KV state rather than recomputing the prefix from tokens [18]. The migration uses the Mooncake [29] backend for high-throughput device-to-device RDMA transfers, avoiding the CPU/TCP data path. This reduces prefix recomputation under a limited memory budget, further improving the system efficiency (see §8.5).

7 Implementation

SpecGen is implemented in approximately 9k lines of Python code. SpecController monitors the LLM’s reasoning trace through the OpenAI API v2.16.0. SpecController parses the reasoning trace to detect trigger signals with a regular expression parser built on 38,745 reasoning traces from GLM-5.1 and DeepSeek-V4-Pro. ElasticScheduler runs as a microservice to manage the validation and profiling requests. Validation workers compile each candidate kernel via nvcc driven

Table 3. Short names of ten KernelBench Level 2/3 kernel optimization tasks in our extended experiments.

Abbr.	Task	Abbr.	Task
T11	Gemm $\times$ LeakyReLU	T16	Conv2d-BN-Scale
T12	Gemm Div-Sum-Scale	T17	Gemm-Add-ReLU
T13	Gemm-Scale-BN	T18	Matmul-GELU-Softmax
T14	Conv2d-Act-BN	T19	MLP (Level 3)
T15	Matmul-Sigmoid-Sum	T20	ReLU Self-Attn (Level 3)

by Ninja 1.13.0 with MAX_JOBS=20, then run a single correctness check against the KernelBench reference on the validation GPU. Profiling workers invoke NCU 2025.1.0 for hardware-counter collection. Both validation and profiling workers run exclusively on GPUs. The system exposes interfaces for user-defined search algorithms, prompt templates, and termination criteria.

8 Evaluation

Our evaluation answers eight questions. **Q1 (§8.2):** How much does SpecGen reduce E2E execution time compared with other systems? **Q2 (§8.3):** How much does SpecGen increase profiling feedback? **Q3 (§8.4):** Does SpecGen keep validation/profiling GPUs busy during the generation phase? **Q4 (§8.5):** Which components contribute to the E2E speedup? **Q5 (§8.6):** Does speculative generation improve final kernel performance rather than trading performance for shorter iterations? **Q6 (§8.7):** What additional token overhead does SpecGen introduce? **Q7 (§8.8):** Does SpecGen remain effective on harder KernelBench Level 2/3 tasks? **Q8 (§8.9):** How do E2E time and final kernel performance change with different termination criteria?

8.1 Experimental Setup

Testbed & Models. Our experiments use up to 18 H200 GPUs in our internal cluster. Each node is interconnected via NVLink and RoCEv2 and hosts an Intel Xeon Platinum 8558 CPU. We serve GLM-5.1 via vLLM with 8 GPUs, while the remaining GPUs are dedicated to validation and profiling. DeepSeek-V4-Pro is accessed via its official API with high reasoning effort. The software stack includes PyTorch 2.10, CUDA 12.9, NCU 2025.1.0, and Ninja. For both models, temperature is set to 0.1 to maintain sharp code-generation formats, and speculative forks disable the reasoning traces. **Workloads & Metrics.** We evaluate the ten KernelBench Level 1 tasks (Table 1) and ten Level 2/3 tasks (Table 3) with a budget of 100 iterations per task. To filter device noise, each kernel’s speedup over the PyTorch reference is averaged across 40 timed runs (following 10 warmups). All cross-task aggregated results report the geometric mean.

Baseline. We compare SpecGen against three representative agentic kernel optimization systems. **(1) CudaForge** [40] is a training-free multi-agent framework that employs a Coder-Judge workflow to iteratively generate and optimize CUDA kernels, guided by hardware feedback from NCU metrics.

(2) AlphaEvolve [27] is a coding agent from Google DeepMind that pairs reasoning LLMs with automated evaluators, using an evolutionary algorithm to iteratively discover and refine kernel performance. **(3) KernelAgent** [26] is an autonomous kernel generation system from Meta. It combines problem analysis, parallel LLM-assisted kernel generation with numerical verification, and a hardware-guided iterative optimization loop to tune kernel performance.

8.2 End-to-End Execution Time

SpecGen consistently reduces E2E execution time and provides final kernels with higher speedups than the three agentic kernel optimization baselines (see Figure 13).

Overall time speedup. Figure 10 shows that across ten KernelBench tasks, SpecGen achieves a geomean time speedup of 1.50 $\times$ over CudaForge on GLM-5.1 and 1.88 $\times$ on DeepSeek-V4-Pro. Against AlphaEvolve, the geomean speedups are 1.68 $\times$ on GLM-5.1 and 1.96 $\times$ on DeepSeek-V4-Pro. Against KernelAgent, the geomean speedups are 1.69 $\times$ and 1.96 $\times$, respectively. Across both models, SpecGen improves end-to-end time by 1.68 $\times$ over CudaForge, 1.81 $\times$ over AlphaEvolve, and 1.82 $\times$ over KernelAgent in geomean.

Long generation latency prolongs iteration time. CudaForge follows an iterative Coder-Judge workflow that waits for each reasoning generation to finish before validation and profiling can provide hardware feedback. AlphaEvolve uses an evolutionary loop with automated evaluators, but each program variant still becomes actionable only after the LLM finishes producing it. KernelAgent adds static analysis, parallel kernel generation, strict numerical verification, and hardware-guided optimization, but its validation and profiling pipeline also depends on completed candidate kernels. As a result, these systems mainly improve the search policy after candidates exist, while validation/profiling resources remain idle during long reasoning generations.

Early termination reduces generation latency. SpecGen changes the timing of candidate availability. SpecController forks non-reasoning generations from reasoning prefixes while the reasoning generation continues as a fallback. When a speculative kernel satisfies the termination criterion, SpecGen terminates the reasoning generation early and advances the iteration. This mechanism directly reduces the longest phase of each iteration.

8.3 Profiling Feedback

SpecGen substantially raises profiling feedback through two effects: it produces more candidate kernels per iteration, and it conditions non-reasoning generations on reasoning prefixes to improve kernel performance.

Overall feedback increment. As shown in Figure 11, on GLM-5.1, SpecGen raises the average profiling feedback to

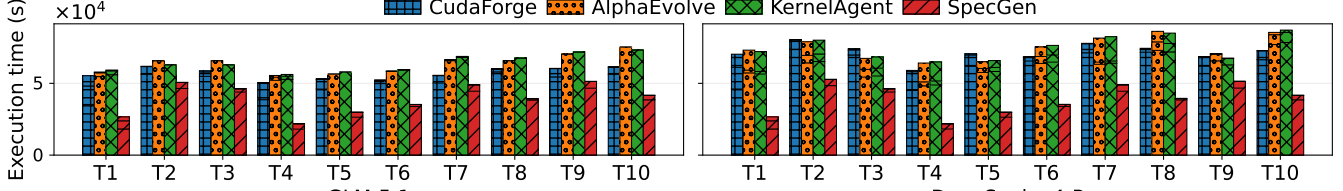


Figure 10. End-to-end execution time of 100 iterations per task on GLM-5.1 and DeepSeek-V4-Pro.

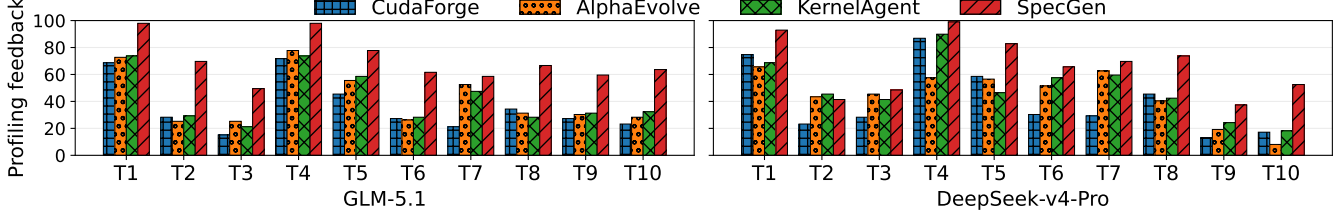


Figure 11. Profiling feedback of 100 iterations per task on GLM-5.1 and DeepSeek-V4-Pro.

70.3, compared with 36.3 for CudaForge, 42.5 for AlphaEvolve, and 42.4 for KernelAgent. This corresponds to geomean lifts of 2.13 $\times$, 1.78 $\times$, and 1.77 $\times$, respectively. On DeepSeek-V4-Pro, SpecGen reaches 66.4, compared with 40.7 for CudaForge, 45.1 for AlphaEvolve, and 49.4 for KernelAgent. The corresponding geomean lifts are 1.84 $\times$, 1.61 $\times$, and 1.40 $\times$. Across both models, SpecGen increases profiling feedback by 1.98 $\times$ over CudaForge, 1.69 $\times$ over AlphaEvolve, and 1.58 $\times$ over KernelAgent in geomean.

More candidates per iteration. CudaForge, AlphaEvolve, and KernelAgent differ in search policy, but all three must wait for completed candidate kernels before validation and profiling can return feedback. CudaForge uses hardware-feedback-guided agent iteration, AlphaEvolve uses evolutionary program refinement, and KernelAgent combines analysis, parallel generation, verification, and hardware-guided tuning. These mechanisms improve how candidates are selected and refined after they exist, but they do not expose validation/profiling to useful kernels during the reasoning generation. SpecGen instead forks speculative non-reasoning generations within the same iteration, so validation sees more candidate kernels under the same iteration budget. This increases the chance that at least one candidate passes validation and reaches profiling.

Faster speculative candidates. SpecGen conditions each speculative generation on the reasoning prefix, which passes kernel-design decisions from the reasoning trace into the non-reasoning prompt. This prefix conditioning improves the performance of speculative kernels and raises the probability that each additional candidate reaches profiling. The combination of more and faster candidates improves profiling feedback across baselines and models.

8.4 Validation/Profiling Resource Utilization

SpecGen substantially improves validation/profiling resource utilization with SpecController and ElasticScheduler. **Overall utilization lift.** Table 4 shows that the three baseline

Table 4. Validation/profiling resource utilization. Utilization is the percentage of E2E time during which resources are busy. ES refers to ElasticScheduler. CF: CudaForge, AE: AlphaEvolve, KA: KernelAgent, **SKG: SpecGen**.

Models	CF	AE	KA	SKG w/o ES	SKG
GLM	4.7%	5.0%	4.2%	56.2%	88.2%
DSv4	11.3%	17.6%	14.2%	74.7%	96.1%

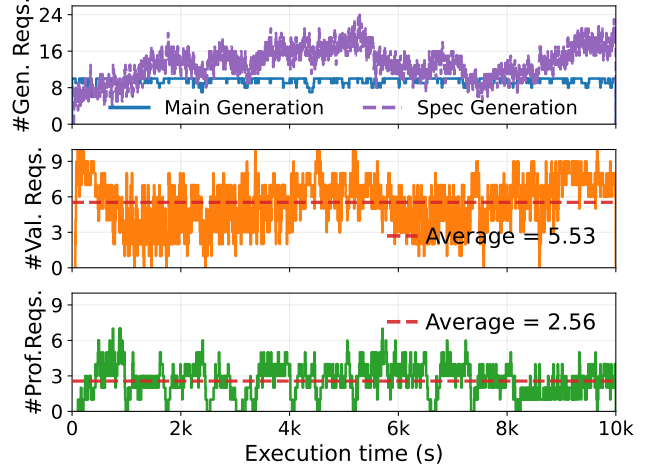


Figure 12. In-flight request counts per phase over the first 10,000 seconds of SpecGen on GLM-5.1.

systems leave most validation/profiling capacity idle. On GLM-5.1, their utilization stays between 4.2% and 5.0%; on DeepSeek-V4-Pro, it stays between 11.3% and 17.6%. SpecGen without ElasticScheduler already raises utilization to 56.2% on GLM-5.1 and 74.7% on DeepSeek-V4-Pro via emitting additional speculative candidate kernels. With ElasticScheduler, utilization further rises to 88.2% and 96.1%, showing that SpecController and ElasticScheduler keep the GPU pool busy under bursty arrivals.

Sporadic validation/profiling requests. Since long reasoning generations dominate iteration time, validation/profiling GPUs wait for completed kernels for most of the run. The feedback bottleneck is amplified because many generated

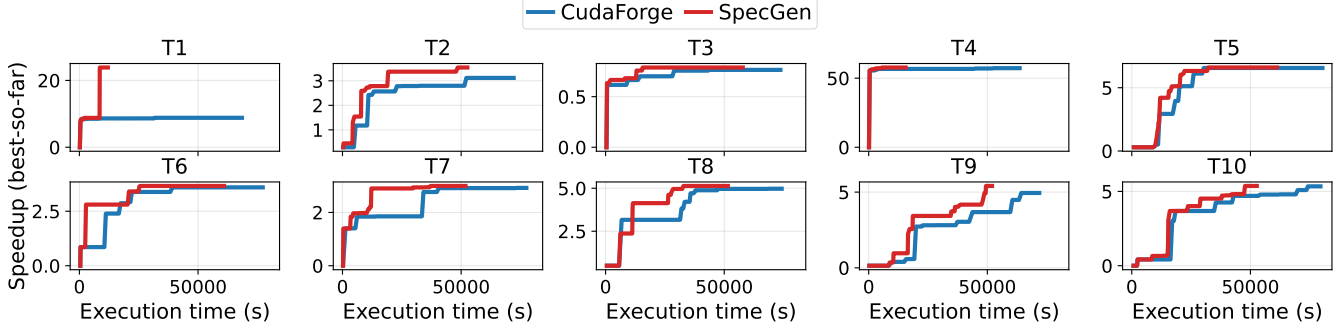


Figure 13. Detailed time-to-speedup over the KernelBench reference across 100 iterations per task on GLM-5.1.

Table 5. Incremental performance breakdown of SpecGen on GLM-5.1 across ten KernelBench tasks. Speedup is geomean end-to-end time speedup over CudaForge.

Configuration	Speedup	Δ
Baseline	1.00×	0.00
+ Speculative Generation	1.46×	0.46↑
+ Resource Reallocation	1.61×	0.15↑
+ Priority Queue	1.69×	0.08↑
+ Remote Prefix Cache	1.77×	0.09↑

kernels fail validation and never reach profiling. Thus, even methods with stronger search logic or parallel candidate generation can leave the validation/profiling GPU pool idle.

More requests and bursty handling. SpecController increases the number of validation/profiling requests by forking speculative generations during the reasoning generation. This alone explains the large jump from baseline utilization to SpecGen without ElasticScheduler. However, speculative requests arrive in bursts, which can leave some GPUs busy while others are idle under a static split. ElasticScheduler reallocates validation/profiling capacity based on queue pressure and adjusts the number of speculative forks when validation/profiling GPUs become idle or busy. Figure 12 illustrates this behavior over the first 10,000 seconds, where validation and profiling remain active instead of waiting for completed reasoning generations. The result is near-saturated validation/profiling GPU utilization.

8.5 Performance Breakdown

Table 5 breaks down where SpecGen’s end-to-end speedup comes from. The first step, speculative generation, accounts for most of the gain by raising geomean speedup from 1.00× to 1.46×. Speculative generation shortens the critical path directly. A speculative kernel can satisfy the termination criterion before the reasoning generation finishes, so the iteration no longer has to wait for the reasoning generation.

The remaining components make those speculative kernels useful sooner. Resource reallocation raises speedup to 1.61× by moving GPUs toward the busier validation or profiling phase, which reduces idle time under bursty speculative arrivals. Priority queues further raise speedup to 1.69× by

Table 6. Best kernel speedup over the KernelBench reference. The first block reports GLM and the second block reports DeepSeek. **X**: no available kernels. CF: CudaForge, AE: AlphaEvolve, KA: KernelAgent, **SKG: SpecGen**.

System	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
CF	8.81	3.12	0.76	57.23	6.56	3.60	2.92	4.97	4.94	5.34
AE	12.83	1.61	X	47.43	2.45	0.81	0.69	4.15	3.98	2.36
KA	21.23	2.21	X	56.98	4.97	2.57	2.60	4.31	5.11	4.67
SKG	23.86	3.54	0.79	57.72	6.60	3.66	2.99	5.13	5.41	5.37
CF	8.64	0.88	0.86	60.87	4.76	3.61	2.30	3.72	0.53	0.46
AE	7.98	0.53	0.61	59.47	1.00	3.36	1.29	3.11	0.82	0.71
KA	6.17	0.23	0.05	61.15	4.98	4.32	2.29	3.50	1.02	0.69
SKG	8.76	1.69	0.90	61.54	5.38	5.94	3.00	3.87	1.19	0.73

validating later-prefix candidates first and returning profiling feedback in FIFO order. Together, these scheduling mechanisms reduce validation/profiling waiting time, which lets SpecController receive results earlier and trigger early termination sooner. Remote prefix caching completes the system by raising speedup to 1.77×. It lets speculative generations reuse the cached reasoning prefix instead of recomputing shared prompt state, so the system can add speculative branches with only modest extra generation cost.

8.6 Kernel Performance

SpecGen improves kernel performance and finds faster kernels within fixed time budget with early termination.

Overall kernel performance. Table 6 shows the best kernel speedup over the KernelBench reference after 100 iterations. On both GLM-5.1 and DeepSeek-V4-Pro, SpecGen obtains the best final kernel on all ten tasks. Its geomean speedup over the reference reaches 5.78× on GLM-5.1 and 3.49× on DeepSeek-V4-Pro. Across both models, this corresponds to geomean lifts of 1.24× over CudaForge, 1.91× over AlphaEvolve, and 1.52× over KernelAgent.

Faster kernel within fixed time budget. In this experiment, SpecGen stops the reasoning generation only after a speculative kernel exceeds the average speedup of previously profiled kernels. Figure 13 shows that with early termination, SpecGen finds faster kernels than CudaForge within the fixed time budget. Across all tasks, the best-so-far speedup trajectory of SpecGen reaches or exceeds the CudaForge

Table 7. Token consumption (millions) of SpecGen relative to CudaForge across 100 iterations on GLM-5.1. Ratio is SpecGen tokens divided by CudaForge tokens.

	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10
CudaForge	1.98	2.42	2.42	2.05	2.64	2.38	2.50	2.41	2.38	2.47
SpecGen	0.89	2.71	2.51	1.06	3.13	2.72	2.51	2.55	2.44	2.62
Ratio	0.45	1.12	1.04	0.52	1.18	1.14	1.01	1.06	1.02	1.06

Table 8. Results on T11–T20 (Level 2/3) with DeepSeek. Each task runs 100 iterations. CF: CudaForge, SKG: **SpecGen**.

Task	E2E (k s)		Prof. FB		Util. (%)		Speedup		Tokens (M)	
	CF	SKG	CF	SKG	CF	SKG	CF	SKG	CF	SKG
T11	37.6	24.5	21.2	41.3	13.6	94.3	0.60	1.25	1.20	1.27
T12	32.2	21.0	1.0	13.2	14.9	81.2	0.01	0.42	0.92	1.45
T13	39.3	21.5	22.2	63.6	14.5	84.8	0.63	0.63	1.40	1.12
T14	34.4	22.6	37.4	51.5	19.2	91.2	1.65	1.68	1.01	1.22
T15	40.2	18.3	29.3	69.7	15.2	86.3	0.74	0.77	1.19	1.96
T16	35.8	17.0	37.4	73.7	19.2	92.4	1.06	1.27	1.27	1.18
T17	38.3	22.6	24.2	55.6	15.2	88.5	0.62	0.74	1.14	1.55
T18	32.7	27.7	7.1	31.3	8.5	81.1	0.35	55.79	1.18	1.10
T19	61.5	49.0	41.9	69.2	22.3	93.2	0.64	1.05	2.94	3.17
T20	55.1	47.2	39.4	77.1	14.0	89.2	1.10	1.39	2.61	2.92

trajectory by the end of the execution. The final results in Table 6 further confirm that speculative generation and early termination improve kernel performance while reducing the end-to-end execution time.

8.7 Overhead of SpecGen

SpecGen introduces modest token overhead with prefix caching and early termination.

Overall token cost. Table 7 shows that SpecGen’s total token consumption is slightly lower than CudaForge despite launching speculative generations. Across all ten tasks, CudaForge consumes 23.66M tokens, while SpecGen consumes 23.14M tokens, or 0.98× of CudaForge. Most matmul tasks incur modest extra tokens, with ratios between 1.01× and 1.18×. However, HingeLoss (T1) and diagonal matmul (T4) terminate early in many iterations. Their reasoning generations are therefore cut short, saving most of the tokens that would otherwise be spent after the useful speculative kernel has already appeared. As a result, T1 and T4 consume only 0.45× and 0.52× of CudaForge’s tokens, which offsets the modest overhead on harder tasks.

Prefix caching and early termination offset token overhead. SpecGen concatenates the original prompt with the current reasoning prefix to form the speculative prompt. Therefore, the prefill of speculative generations can reuse the KV cache of the reasoning generation. Moreover, early termination also saves tokens. Both effects offset the modest overhead across these ten tasks.

8.8 Performance on Harder Tasks

SpecGen remains effective on harder KernelBench Level 2/3 tasks, improving runtime, feedback density, validation and profiling resource utilization, and final kernel speedup over

the reference with modest token overhead. Table 8 compares SpecGen with CudaForge on ten harder tasks with DeepSeek. On average, SpecGen reduces E2E time by 1.57× and raises profiling feedback from 26.1 to 54.6. It also increases resource utilization from 15.7% to 88.2%. Moreover, SpecGen finds kernels at least as fast as CudaForge on all ten tasks and a strictly faster kernel on nine of them, improving geomean kernel speedup over the reference from 0.49× to 1.42×. The additional token cost remains moderate, with a 14.0% increase in total token consumption.

8.9 Termination Criterion Analysis

SpecGen improves kernel performance within a fixed token budget and exposes a tunable trade-off between E2E time and kernel performance.

More feedback per token yields faster kernels. Table 9 compares SpecGen against CudaForge under matched token budgets. Simply doubling CudaForge’s token budget barely improves performance, raising the final speedup from 5.05× to 5.15× on GLM and from 2.61× to 2.74× on DeepSeek. SpecGen with the historical-average criterion reaches 5.78× and 3.18× while consuming only 130.2% of CudaForge’s tokens, fewer than CudaForge+. The reason is that speculative generations are non-reasoning and reuse the cached reasoning prefix, so each candidate costs few tokens. Under the same budget, SpecGen therefore produces far more profiling feedback (114.9 and 101.1 additional feedback) than extra reasoning generations would. This added feedback in return guides the LLM toward faster kernels.

The termination threshold tunes the time-performance trade-off. Lowering the threshold trades kernel performance for shorter runtime. First-valid termination stops earliest, giving the shortest E2E time, yet still emits many candidate kernels and reaches 5.53× and 2.95×, already faster than CudaForge. The historical-average default sits in the middle, preserving most of the latency benefit while improving performance. When tokens are abundant, disabling termination finds the fastest kernels at the cost of more tokens. Across every criterion, SpecGen produces faster kernels than CudaForge under a comparable or smaller token budget.

9 Related Work

Training-based kernel optimization. Recent work improves CUDA and Triton kernel generation by adapting the *model* itself through reinforcement learning and post-training over execution feedback. Kevin [2], CUDA-L1 [22], and CUDA Agent [7] define the basic recipe: optimize the LLM on execution outcomes, pairwise fast-vs.-slow comparisons, or large-scale agentic RL curricula to raise intrinsic kernel performance. Later systems broaden this recipe with memory-augmented in-context RL and cross-task reuse [12],

Table 9. Comparison of CudaForge and SpecGen with different termination criteria across T1–T10. The first block reports results with GLM and the second reports DeepSeek. CudaForge⁺: Doubled token budget for CudaForge.

Method	#Additional feedback	Kernel Speedup	Token Ratio	#Term.	E2E time (s)
CudaForge	0	5.05×	100%	0	73.9k
CudaForge ⁺	35.2	5.15×	208.6%	0	75.2k
First valid	70.0	5.53×	114.3%	70.3	39.8k
Hist. avg.	114.9	5.78×	130.2%	63.0	43.8k
Hist. best	535.9	6.06×	229.5%	9.0	70.3k
No term.	587.2	6.15×	240.0%	0	73.9k
CudaForge	0	2.61×	100%	0	57.0k
CudaForge ⁺	44.3	2.74×	210.9%	0	57.7k
First valid	64.5	2.95×	105.1%	64.5	29.6k
Hist. avg.	101.1	3.18×	130.2%	52.8	35.9k
Hist. best	252.3	4.79×	208.3%	8.1	54.6k
No term.	269.1	5.08×	218.1%	0	57.0k

hierarchical strategy decomposition [38, 42], stronger training environments and synthetic curricula [15, 25], and frontier-model fine-tuning [35]. The common pattern is to spend additional training compute and infrastructure to improve the generator itself. SpecGen is complementary: we leave the LLM unchanged and instead reorganize how the agent loop consumes its outputs, so the gains from better-trained kernel models and from our runtime can compose rather than substitute for one another.

Speculative decoding. Speculative decoding [6, 19] accelerates a *single* LLM generation by drafting several future tokens and verifying them in parallel. Medusa [4] removes the separate draft model with lightweight decoding heads, and later variants explore feature-level drafting, self-speculation, adaptive depth, and simpler draft architectures [1, 3, 23, 24, 39, 41]. All of these methods remain strictly *token-level*, speculatively generating tokens to raise throughput. SpecGen instead operates at the *iteration-level*, speculatively generating kernels to raise the kernel count per iteration. The two forms of speculation are orthogonal.

10 Conclusion

This paper presents SpecGen, an efficient agentic kernel optimization system. We propose *speculative generation* to address the system-level inefficiencies of agentic kernel optimization. We design SpecController and ElasticScheduler to manage the speculative generations and validation/profiling resources, respectively. Extensive experiments show the effectiveness of SpecGen in improving the agentic kernel optimization efficiency against baseline systems.

References

[1] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *CoRR*, abs/2402.05109, 2024.

[2] Carlo Baronio, Pietro Marsella, Ben Pan, Simon Guo, and Silas Alberti. Kevin: Multi-turn RL for generating CUDA kernels. *CoRR*, abs/2507.11948, 2025.

[3] Oscar Brown, Zhengjie Wang, Andrea Do, Nikhil Mathew, and Cheng Yu. Dynamic depth decoding: Faster speculative decoding for llms. *CoRR*, abs/2409.00142, 2024.

[4] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21–27, 2024*, Proceedings of Machine Learning Research, pages 5209–5235. PMLR / OpenReview.net, 2024.

[5] Shiyi Cao, Ziming Mao, Joseph E. Gonzalez, and Ion Stoica. K-search: LLM kernel generation via co-evolving intrinsic world model. *CoRR*, abs/2602.19128, 2026.

[6] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *CoRR*, abs/2302.01318, 2023.

[7] Weinan Dai, Hanlin Wu, Qiying Yu, Huan-Ang Gao, Jiahao Li, Chengquan Jiang, Weiqiang Lou, Yufan Song, Hongli Yu, Jiaze Chen, Wei-Ying Ma, Ya-Qin Zhang, Jingjing Liu, Mingxuan Wang, Xin Liu, and Hao Zhou. CUDA agent: Large-scale agentic RL for high-performance CUDA kernel generation. *CoRR*, abs/2602.24286, 2026.

[8] Tri Dao. Flashattention-2: Faster attention with better parallelism and work partitioning. 2024.

[9] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.

[10] DeepSeek-AI. DeepSeek-V4: Towards highly efficient million-token context intelligence. <https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro>, 2026. Technical Report available at https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf.

[11] Juncheng Dong, Yang Yang, Tao Liu, Yang Wang, Feng Qi, Vahid Tarokh, Kaushik Rangadurai, and Shuang Yang. STARK: strategic team of agents for refining kernels. *CoRR*, abs/2510.16996, 2025.

[12] Shengjun Kris Dong, Sahil Modi, Dima Nikiforov, Sana Damani, Edward Lin, Siva Kumar Sastry Hari, and Christos Kozyrakis. Kernelblaster: Continual cross-task CUDA optimization via memory-augmented in-context reinforcement learning. *CoRR*, abs/2602.14293, 2026.

[13] He Du, Qiming Ge, Jiakai Hu, Aijun Yang, Zheng Cai, Zixian Huang, Sheng Yuan, Qinxiu Cheng, Xincheng Xie, Yicheng Chen, Yining Li, Jiaxing Xie, Huanan Dong, Yaguang Wu, Xiangjun Huang, Jian Yang, Hui Wang, Bowen Zhou, Bowen Li, Qipeng Guo, and Kai Chen. Kernel-smith: A unified recipe for evolutionary kernel optimization. *CoRR*, abs/2603.28342, 2026.

[14] GLM. GLM-5: from vibe coding to agentic engineering. *CoRR*, abs/2602.15763, 2026.

[15] Siqi Guo, Ming Lin, and Tianbao Yang. Drtriton: Large-scale synthetic data reinforcement learning for triton kernel generation. *CoRR*, abs/2603.21465, 2026.

[16] Siva Kumar Sastry Hari, Vignesh Balaji, Sana Damani, Qijing Huang, and Christos Kozyrakis. Improving efficiency of GPU kernel optimization agents using a domain-specific language and speed-of-light guidance. *CoRR*, abs/2603.29010, 2026.

[17] Jaber Jaber and Osama Jaber. Autokernel: Autonomous GPU kernel optimization via iterative agent-driven search. *CoRR*, abs/2603.21331, 2026.

- 2026.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In Jason Flinn, Margo I. Seltzer, Peter Druschel, Antoine Kaufmann, and Jonathan Mace, editors, *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pages 611–626. ACM, 2023.
 - [19] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, Proceedings of Machine Learning Research, pages 19274–19286. PMLR, 2023.
 - [20] Haonan Li, Keyu Man, Partha Kanuparth, Hanning Chen, Wei Sun, Sreen Tallam, Chenguang Zhu, Kevin Zhu, and Zhiyun Qian. Tritonforge: Profiling-guided framework for automated triton kernel optimization. *CoRR*, abs/2512.09196, 2025.
 - [21] Jianling Li, Shangzhan Li, Zhenye Gao, Qi Shi, Yuxuan Li, Zefan Wang, Jiacheng Huang, WangHaojie WangHaojie, Jianrong Wang, Xu Han, Zhiyuan Liu, and Maosong Sun. Tritonbench: Benchmarking large language model capabilities for generating triton operators. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, Findings of ACL, pages 23053–23066. Association for Computational Linguistics, 2025.
 - [22] Xiaoya Li, Xiaofei Sun, Albert Wang, Jiwei Li, and Chris Shum. CUDA-L1: improving CUDA optimization via contrastive reinforcement learning. *CoRR*, abs/2507.14111, 2025.
 - [23] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. pages 28935–28948, 2024.
 - [24] Fangcheng Liu, Yehui Tang, Zhenhua Liu, Yunsheng Ni, Duyu Tang, Kai Han, and Yunhe Wang. Kangaroo: Lossless self-speculative decoding for accelerating llms via double early exiting. 2024.
 - [25] Wei Liu, Jiawei Xu, Yingru Li, Longtao Zheng, Tianjian Li, Qian Liu, and Junxian He. Dr. kernel: Reinforcement learning done right for triton kernel generations. *CoRR*, abs/2602.05885, 2026.
 - [26] Meta PyTorch Team. KernelAgent: Autonomous GPU kernel generation & optimization via deep agents, 2025. Blog post: <https://pytorch.org/blog/kernelfalcon-autonomous-gpu-kernel-generation-via-deep-agents/>.
 - [27] Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery. *CoRR*, abs/2506.13131, 2025.
 - [28] Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. Kernelbench: Can llms write efficient GPU kernels? In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu, editors, *Forty-second International Conference on Machine Learning, ICML 2025, Vancouver, BC, Canada, July 13-19, 2025*, Proceedings of Machine Learning Research. PMLR / OpenReview.net, 2025.
 - [29] Ruoyu Qin, Zheming Li, Weiran He, Jiale Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation - A kvcache-centric architecture for serving LLM chatbot. In Haryadi S. Gunawi and Vasily Tarasov, editors, *23rd USENIX Conference on File and Storage Technologies, FAST 2025, Santa Clara, CA, February 25-27, 2025*, pages 155–170. USENIX Association, 2025.
 - [30] Tara Saba, Anne Ouyang, Xujie Si, and Fan Long. Cutegen: An llm-based agentic framework for generation and optimization of high-performance GPU kernels using cute. *CoRR*, abs/2604.01489, 2026.
 - [31] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Raman, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024.
 - [32] Asankhaya Sharma. Openevolve: an open-source evolutionary coding agent, 2025.
 - [33] Qitong Sun, Jun Han, Tianlin Li, Zhe Tang, Sheng Chen, Fei Yang, Aishan Liu, Xianglong Liu, and Yang Liu. KernelSkill: A multi-agent framework for GPU kernel optimization. *CoRR*, abs/2603.10085, 2026.
 - [34] KernelEvolve Team and Meta Platforms. Kernelevolve: Scaling agentic kernel coding for heterogeneous AI accelerators at meta. *CoRR*, abs/2512.23236, 2025.
 - [35] Ali Tehrani, Yahya Emara, Essam Wissam, Wojciech Paluch, Waleed Atallah, Lukasz Dudziak, and Mohamed S. Abdelfattah. Fine-tuning GPT-5 for GPU kernel generation. *CoRR*, abs/2602.11000, 2026.
 - [36] Arya Tschand, Muhammad A. Awad, Ryan Swann, Kesavan Ramakrishnan, Jeffrey Jian Ma, Keith Lowery, Ganesh Dasika, and Vijay Janapa Reddi. Swizzleperf: Hardware-aware llms for GPU kernel performance optimization. *CoRR*, abs/2508.20258, 2025.
 - [37] Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. Astra: A multi-agent system for GPU kernel performance optimization. *CoRR*, abs/2509.07506, 2025.
 - [38] Jiin Woo, Shaowei Zhu, Allen Nie, Zhen Jia, Yida Wang, and Youngsuk Park. Tritonrl: Training llms to think and code triton without cheating. *CoRR*, abs/2510.17891, 2025.
 - [39] Heming Xia, Yongqi Li, Jun Zhang, Cunxiao Du, and Wenjie Li. SWIFT: on-the-fly self-speculative decoding for LLM inference acceleration. 2025.
 - [40] Zijian Zhang, Rong Wang, Shiyang Li, Yuebo Luo, Mingyi Hong, and Caiwen Ding. Cudaforge: An agent framework with hardware feedback for CUDA kernel optimization. *CoRR*, abs/2511.01884, 2025.
 - [41] Wei Zhong, Manasa Bharadwaj, Yixiao Wang, Nikhil Verma, Yipeng Ji, and Chul Lee. Cross-attention speculative decoding. *CoRR*, abs/2505.24544, 2025.
 - [42] Xinguo Zhu, Shaohui Peng, Jiaming Guo, Yunji Chen, Qi Guo, Yuanbo Wen, Hang Qin, Ruizhi Chen, Qirui Zhou, Ke Gao, Yanjun Wu, Chen Zhao, and Ling Li. Qimeng-kernel: Macro-thinking micro-coding paradigm for llm-based high-performance GPU kernel generation. pages 29168–29176, 2026.